

Assignment No. 12

- **TITLE** – Queues are essential in real-world systems like printer spooling or traffic management. Design a menu-driven program to implement and explore different types of queues: Linear Queue, Circular Queue, Dequeue (input/output restricted), Priority Queue (ascending/descending), and Linked Queue. Understand how each queue type handles different operational constraints and efficiently manages data flow in Printers or Traffic Management.

➤ **THEORY** –

Queues are fundamental data structures that follow the First-In-First-Out (FIFO) principle. They are essential in systems where order preservation and fair resource allocation are critical, such as printer spooling, traffic management, and task scheduling.

Types of Queues and Their Real-World Applications

1. Linear Queue

- Structure: Simple sequential storage with front and rear pointers
- Real-World Use: Basic printer spooling systems
- Limitation: Memory wastage due to inability to reuse dequeued spaces

2. Circular Queue

- Structure: Circular array that reuses memory space
- Real-World Use:
 - Traffic light sequencing
 - CPU scheduling
 - Circular buffers in I/O systems
- Advantage: Efficient memory utilization

3. Deque (Double-Ended Queue)

- Structure: Allows insertion and deletion from both ends
- Types:
 - Input Restricted: Insertion only at one end
 - Output Restricted: Deletion only at one end
- Real-World Use:
 - Web browser history (back/forward)
 - Undo-redo operations
 - Job scheduling with priority interrupts

4. Priority Queue

- Structure: Elements processed based on priority rather than arrival time
- Types:
 - Ascending: Lower numbers = higher priority
 - Descending: Higher numbers = higher priority

- Real-World Use:
 - Emergency services dispatch
 - Hospital triage systems
 - Operating system process scheduling

5. Linked Queue

- Structure: Dynamic memory allocation using linked list
- Real-World Use:
 - Message queues in networking
 - Dynamic task scheduling
 - Unlimited size applications

Algorithm Analysis

1. Linear Queue Operations

Enqueue Algorithm:

Algorithm enqueue(value):

1. IF rear = MAX_SIZE - 1:
 PRINT "Queue Overflow"
 RETURN
2. IF front = -1:
 front ← 0
3. rear ← rear + 1
4. arr[rear] ← value
5. END

Dequeue Algorithm:

Algorithm dequeue():

1. IF front = -1 OR front > rear:
 PRINT "Queue Underflow"
 RETURN
2. value ← arr[front]
3. front ← front + 1
4. IF front > rear:
 front ← rear ← -1
5. RETURN value

2. Circular Queue Operations

Enqueue Algorithm:

Algorithm enqueue(value):

1. IF (front = 0 AND rear = MAX_SIZE-1) OR (rear = (front-1) mod (MAX_SIZE-1)):
 PRINT "Queue Full"
 RETURN
2. IF front = -1:
 front ← rear ← 0
3. ELSE IF rear = MAX_SIZE-1 AND front ≠ 0:

```

    rear ← 0
4. ELSE:
    rear ← rear + 1
5. arr[rear] ← value
6. END

```

Deque Algorithm:

Algorithm dequeue():

```

1. IF front = -1:
    PRINT "Queue Empty"
    RETURN
2. value ← arr[front]
3. IF front = rear:
    front ← rear ← -1
4. ELSE IF front = MAX_SIZE-1:
    front ← 0
5. ELSE:
    front ← front + 1
6. RETURN value

```

Priority Queue Operations

Enqueue with Priority Algorithm:

Algorithm priorityEnqueue(value):

```

1. IF size = MAX_SIZE:
    PRINT "Queue Full"
    RETURN
2. IF size = 0:
    arr[0] ← value
3. ELSE:
    i ← size - 1
    WHILE i >= 0 AND ((ascending AND arr[i] > value) OR (!ascending AND arr[i] < value)):
        arr[i+1] ← arr[i]
        i ← i - 1
    arr[i+1] ← value
4. size ← size + 1
5. END

```

Deque Operations

Insert at Front Algorithm:

Algorithm insertFront(value):

```

1. IF isFull():
    PRINT "Deque Full"
    RETURN
2. IF front = -1:
    front ← rear ← 0
3. ELSE IF front = 0:
    front ← MAX_SIZE - 1

```

4. ELSE:

front $\leftarrow$ front - 1

5. arr[front] $\leftarrow$ value

6. END

➤ **CODE –**

```
#include <iostream>
#include <queue>
using namespace std;

const int MAX_SIZE = 5;

class LinearQueue {
private:
    int arr[MAX_SIZE];
    int front, rear;

public:
    LinearQueue() {
        front = -1;
        rear = -1;
    }

    bool isFull() {
        return rear == MAX_SIZE - 1;
    }

    bool isEmpty() {
        return front == -1 || front > rear;
    }

    void enqueue(int value) {
        if (isFull()) {
            cout << "Queue is full!" << endl;
            return;
        }
        if (front == -1) front = 0;
        arr[++rear] = value;
        cout << "Enqueued: " << value << endl;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return;
        }
        cout << "Dequeued: " << arr[front++] << endl;
        if (front > rear) {
            front = rear = -1;
        }
    }

    void display() {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return;
        }
        cout << "Linear Queue: ";
```

```

        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

class CircularQueue {
private:
    int arr[MAX_SIZE];
    int front, rear;

public:
    CircularQueue() {
        front = -1;
        rear = -1;
    }

    bool isFull() {
        return (front == 0 && rear == MAX_SIZE - 1) || (rear == (front - 1) %
(MAX_SIZE - 1));
    }

    bool isEmpty() {
        return front == -1;
    }

    void enqueue(int value) {
        if (isFull()) {
            cout << "Queue is full!" << endl;
            return;
        }
        if (front == -1) {
            front = rear = 0;
        } else if (rear == MAX_SIZE - 1 && front != 0) {
            rear = 0;
        } else {
            rear++;
        }
        arr[rear] = value;
        cout << "Enqueued: " << value << endl;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Queue is empty!" << endl;
            return;
        }
        cout << "Dequeued: " << arr[front] << endl;
        if (front == rear) {
            front = rear = -1;
        } else if (front == MAX_SIZE - 1) {
            front = 0;
        } else {
            front++;
        }
    }

    void display() {
        if (isEmpty()) {

```

```

        cout << "Queue is empty!" << endl;
        return;
    }
    cout << "Circular Queue: ";
    if (rear >= front) {
        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }
    } else {
        for (int i = front; i < MAX_SIZE; i++) {
            cout << arr[i] << " ";
        }
        for (int i = 0; i <= rear; i++) {
            cout << arr[i] << " ";
        }
    }
    cout << endl;
}
};

class Deque {
private:
    int arr[MAX_SIZE];
    int front, rear;

public:
    Deque() {
        front = -1;
        rear = -1;
    }

    bool isFull() {
        return (front == 0 && rear == MAX_SIZE - 1) || (front == rear + 1);
    }

    bool isEmpty() {
        return front == -1;
    }

    void insertFront(int value) {
        if (isFull()) {
            cout << "Deque is full!" << endl;
            return;
        }
        if (front == -1) {
            front = rear = 0;
        } else if (front == 0) {
            front = MAX_SIZE - 1;
        } else {
            front--;
        }
        arr[front] = value;
        cout << "Inserted at front: " << value << endl;
    }

    void insertRear(int value) {
        if (isFull()) {
            cout << "Deque is full!" << endl;
            return;
        }
    }

```

```

        if (front == -1) {
            front = rear = 0;
        } else if (rear == MAX_SIZE - 1) {
            rear = 0;
        } else {
            rear++;
        }
        arr[rear] = value;
        cout << "Inserted at rear: " << value << endl;
    }

    void deleteFront() {
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return;
        }
        cout << "Deleted from front: " << arr[front] << endl;
        if (front == rear) {
            front = rear = -1;
        } else if (front == MAX_SIZE - 1) {
            front = 0;
        } else {
            front++;
        }
    }

    void deleteRear() {
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return;
        }
        cout << "Deleted from rear: " << arr[rear] << endl;
        if (front == rear) {
            front = rear = -1;
        } else if (rear == 0) {
            rear = MAX_SIZE - 1;
        } else {
            rear--;
        }
    }

    void display() {
        if (isEmpty()) {
            cout << "Deque is empty!" << endl;
            return;
        }
        cout << "Deque: ";
        int i = front;
        while (true) {
            cout << arr[i] << " ";
            if (i == rear) break;
            i = (i + 1) % MAX_SIZE;
        }
        cout << endl;
    }
};

class PriorityQueue {
private:
    int arr[MAX_SIZE];

```

```

int size;
bool ascending;

public:
    PriorityQueue(bool isAscending = true) {
        size = 0;
        ascending = isAscending;
    }

    bool isFull() {
        return size == MAX_SIZE;
    }

    bool isEmpty() {
        return size == 0;
    }

    void enqueue(int value) {
        if (isFull()) {
            cout << "Priority Queue is full!" << endl;
            return;
        }
        if (isEmpty()) {
            arr[0] = value;
        } else {
            int i;
            for (i = size - 1; i >= 0; i--) {
                if ((ascending && arr[i] > value) || (!ascending && arr[i] < value)) {
                    arr[i + 1] = arr[i];
                } else {
                    break;
                }
            }
            arr[i + 1] = value;
        }
        size++;
        cout << "Enqueued with priority: " << value << endl;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Priority Queue is empty!" << endl;
            return;
        }
        cout << "Dequeued: " << arr[0] << endl;
        for (int i = 0; i < size - 1; i++) {
            arr[i] = arr[i + 1];
        }
        size--;
    }

    void display() {
        if (isEmpty()) {
            cout << "Priority Queue is empty!" << endl;
            return;
        }
        cout << "Priority Queue (" << (ascending ? "Ascending" : "Descending") << "):";

        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";

```



```

    }
    cout << endl;
}
};

struct Node {
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = nullptr;
    }
};

class LinkedQueue {
private:
    Node* front;
    Node* rear;

public:
    LinkedQueue() {
        front = rear = nullptr;
    }

    bool isEmpty() {
        return front == nullptr;
    }

    void enqueue(int value) {
        Node* newNode = new Node(value);
        if (rear == nullptr) {
            front = rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }
        cout << "Enqueued: " << value << endl;
    }

    void dequeue() {
        if (isEmpty()) {
            cout << "Linked Queue is empty!" << endl;
            return;
        }
        Node* temp = front;
        cout << "Dequeued: " << front->data << endl;
        front = front->next;
        if (front == nullptr) {
            rear = nullptr;
        }
        delete temp;
    }

    void display() {
        if (isEmpty()) {
            cout << "Linked Queue is empty!" << endl;
            return;
        }
        cout << "Linked Queue: ";
    }
};

```

```

        Node* temp = front;
        while (temp != nullptr) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};

int main() {
    int mainChoice, queueChoice;
    LinearQueue lq;
    CircularQueue cq;
    Deque dq;
    PriorityQueue ascPQ(true);
    PriorityQueue descPQ(false);
    LinkedQueue linkedQ;

    do {
        cout << "\n=== QUEUE MANAGEMENT SYSTEM ===" << endl;
        cout << "1. Linear Queue (Printer Spooling)" << endl;
        cout << "2. Circular Queue (Traffic Management)" << endl;
        cout << "3. Deque (Input/Restricted Operations)" << endl;
        cout << "4. Priority Queue (Emergency Services)" << endl;
        cout << "5. Linked Queue (Dynamic Memory)" << endl;
        cout << "6. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> mainChoice;

        if (mainChoice == 6) break;

        do {
            cout << "\n--- Queue Operations ---" << endl;
            cout << "1. Enqueue/Insert" << endl;
            cout << "2. Dequeue/Delete" << endl;
            cout << "3. Display Queue" << endl;
            if (mainChoice == 3) {
                cout << "4. Insert at Front" << endl;
                cout << "5. Insert at Rear" << endl;
                cout << "6. Delete from Front" << endl;
                cout << "7. Delete from Rear" << endl;
                cout << "8. Back to Main Menu" << endl;
            } else if (mainChoice == 4) {
                cout << "4. Use Ascending Priority" << endl;
                cout << "5. Use Descending Priority" << endl;
                cout << "6. Back to Main Menu" << endl;
            } else {
                cout << "4. Back to Main Menu" << endl;
            }
            cout << "Enter operation choice: ";
            cin >> queueChoice;

            int value;
            switch (mainChoice) {
                case 1: // Linear Queue
                    switch (queueChoice) {
                        case 1:
                            cout << "Enter value to enqueue: ";
                            cin >> value;
                            lq.enqueue(value);

```

```

        break;
    case 2:
        lq.dequeue();
        break;
    case 3:
        lq.display();
        break;
    case 4:
        break;
    default:
        cout << "Invalid choice!" << endl;
}
break;

case 2: // Circular Queue
switch (queueChoice) {
    case 1:
        cout << "Enter value to enqueue: ";
        cin >> value;
        cq.enqueue(value);
        break;
    case 2:
        cq.dequeue();
        break;
    case 3:
        cq.display();
        break;
    case 4:
        break;
    default:
        cout << "Invalid choice!" << endl;
}
break;

case 3: // Deque
switch (queueChoice) {
    case 1:
    case 5:
        cout << "Enter value to insert at rear: ";
        cin >> value;
        dq.insertRear(value);
        break;
    case 2:
    case 6:
        dq.deleteFront();
        break;
    case 3:
        dq.display();
        break;
    case 4:
        cout << "Enter value to insert at front: ";
        cin >> value;
        dq.insertFront(value);
        break;
    case 7:
        dq.deleteRear();
        break;
    case 8:
        break;
    default:

```

```

        cout << "Invalid choice!" << endl;
    }
    break;

case 4: // Priority Queue
    switch (queueChoice) {
        case 1:
            cout << "Enter value to enqueue: ";
            cin >> value;
            if (ascPQ.isEmpty() && descPQ.isEmpty()) {
                ascPQ.enqueue(value);
            } else if (!ascPQ.isEmpty()) {
                ascPQ.enqueue(value);
            } else {
                descPQ.enqueue(value);
            }
            break;
        case 2:
            if (!ascPQ.isEmpty()) {
                ascPQ.dequeue();
            } else if (!descPQ.isEmpty()) {
                descPQ.dequeue();
            }
            break;
        case 3:
            if (!ascPQ.isEmpty()) {
                ascPQ.display();
            } else if (!descPQ.isEmpty()) {
                descPQ.display();
            } else {
                cout << "Priority Queue is empty!" << endl;
            }
            break;
        case 4:
            cout << "Switched to Ascending Priority" << endl;
            while (!descPQ.isEmpty()) {
                int val = 0;
                ascPQ.enqueue(val);
                descPQ.dequeue();
            }
            break;
        case 5:
            cout << "Switched to Descending Priority" << endl;
            while (!ascPQ.isEmpty()) {
                int val = 0;
                descPQ.enqueue(val);
                ascPQ.dequeue();
            }
            break;
        case 6:
            break;
        default:
            cout << "Invalid choice!" << endl;
    }
    break;

case 5: // Linked Queue
    switch (queueChoice) {
        case 1:
            cout << "Enter value to enqueue: ";

```

```

        cin >> value;
        linkedQ.enqueue(value);
        break;
    case 2:
        linkedQ.dequeue();
        break;
    case 3:
        linkedQ.display();
        break;
    case 4:
        break;
    default:
        cout << "Invalid choice!" << endl;
    }
    break;

    default:
        cout << "Invalid main choice!" << endl;
    }
} while ((mainChoice == 3 && queueChoice != 8) ||
        (mainChoice == 4 && queueChoice != 6) ||
        (mainChoice != 3 && mainChoice != 4 && queueChoice != 4));

} while (mainChoice != 6);

cout << "Thank you for using Queue Management System!" << endl;
return 0;
}

```

➤ OUTPUT –

1. Linear Queue

--- Queue Operations ---

1. Enqueue/Insert

2. Dequeue/Delete

3. Display Queue

4. Back to Main Menu

Enter operation choice: 1

Enter value to enqueue: 10

Enqueued: 10

--- Queue Operations ---

1. Enqueue/Insert

2. Dequeue/Delete

3. Display Queue

4. Back to Main Menu

Enter operation choice: 1

Enter value to enqueue: 20

Enqueued: 20

--- Queue Operations ---

1. Enqueue/Insert

2. Dequeue/Delete

3. Display Queue

4. Back to Main Menu

Enter operation choice: 3

Linear Queue: 10 20

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Back to Main Menu
- Enter operation choice: 2
Dequeued: 10

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Back to Main Menu
- Enter operation choice: 3
Linear Queue: 20

2. Circular Queue

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Back to Main Menu
- Enter operation choice: 1
Enter value to enqueue: 10
Enqueued: 10

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Back to Main Menu
- Enter operation choice: 1
Enter value to enqueue: 20
Enqueued: 20

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Back to Main Menu
- Enter operation choice: 2
Dequeued: 10

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Back to Main Menu
- Enter operation choice: 1

Enter value to enqueue: 30
Enqueued: 30

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Back to Main Menu
- Enter operation choice: 3
Circular Queue: 20 30

3. Priority Queue

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Use Ascending Priority
 5. Use Descending Priority
 6. Back to Main Menu
- Enter operation choice: 1
Enter value to enqueue: 10
Enqueued with priority: 10

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Use Ascending Priority
 5. Use Descending Priority
 6. Back to Main Menu
- Enter operation choice: 1
Enter value to enqueue: 20
Enqueued with priority: 20

--- Queue Operations ---

1. Enqueue/Insert
 2. Dequeue/Delete
 3. Display Queue
 4. Use Ascending Priority
 5. Use Descending Priority
 6. Back to Main Menu
- Enter operation choice: 1
Enter value to enqueue: 30
Enqueued with priority: 30

--- Queue Operations ---

1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Use Ascending Priority
5. Use Descending Priority

6. Back to Main Menu
Enter operation choice: 3
Priority Queue (Ascending): 10 20 30

--- Queue Operations ---
1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Use Ascending Priority
5. Use Descending Priority
6. Back to Main Menu
Enter operation choice: 4
Switched to Ascending Priority

--- Queue Operations ---
1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Use Ascending Priority
5. Use Descending Priority
6. Back to Main Menu
Enter operation choice: 3
Priority Queue (Ascending): 10 20 30

4. Linked Queue

--- Queue Operations ---
1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Back to Main Menu
Enter operation choice: 1
Enter value to enqueue: 50
Enqueued: 50

--- Queue Operations ---
1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Back to Main Menu
Enter operation choice: 1
Enter value to enqueue: 60
Enqueued: 60

--- Queue Operations ---
1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Back to Main Menu
Enter operation choice: 1
Enter value to enqueue: 70
Enqueued: 70

--- Queue Operations ---

1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Back to Main Menu

Enter operation choice: 3

Linked Queue: 50 60 70

--- Queue Operations ---

1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Back to Main Menu

Enter operation choice: 2

Dequeued: 50

--- Queue Operations ---

1. Enqueue/Insert
2. Dequeue/Delete
3. Display Queue
4. Back to Main Menu

Enter operation choice: 3

Linked Queue: 60 70

➤ **CONCLUSION –**

This practical lab demonstrates how different types of queues are implemented and how they will be used in various different real world applications.